Section 7 — Tool Calling Deep Dive

How Agents Call External Functions, APIs, and Real-World Services

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

#	Topic
7.1	What Is a Tool
7.2	Why Tools Are the Most Important Part of an Agent
7.3	How Tool Calling Works — Step by Step
7.4	Types of Tools
7.5	Real-World Tools by Industry
7.6	How to Give an Agent Tool Access
7.7	Tool Design Best Practices
7.8	Tool Calling in LangGraph, AutoGen, and CrewAI
7.9	Section Summary

7.0 What Is an API — Before We Talk About Tools

Before understanding tools, you need to understand APIs. Because every tool you build for an agent is built on top of an API.

What Is an API

API stands for **Application Programming Interface**.

An API is a **contract between two software systems** that defines exactly how they can talk to each other — what one system can ask for, and what the other system will send back.

> **In one sentence:** An API is a door between two systems. You knock with a specific request. The system opens the door and hands you back exactly what you asked for.

Two Ways to Interact with a System

Every system — whether it is a bank, a hospital, a food delivery app, or a flight booking platform — has two different types of interfaces depending on **who is talking to it**.

When a Human Wants to Interact — You Need a Frontend

A frontend is the visual layer — the app, the website, the screen that humans see and click on.

```
Human user
    |
    |  opens app, clicks buttons, fills forms
    ▼
FRONTEND
  (Mobile App / Website / Dashboard)
  Zomato App, HDFC Bank App, MakeMyTrip Website
    |
    |  user clicks "Search Flights"
    |  frontend collects the input
    ▼
BACKEND / API
  (processes the request, talks to database)
    |
    ▼
DATABASE / EXTERNAL SYSTEM
  (flights, prices, inventory)
```

The frontend is designed for humans. It has buttons, colours, forms, images. A human cannot directly talk to the backend — they go through the frontend.

When One System Wants to Talk to Another System — You Need an API

When your agent (or any software) wants to talk to an external system, there is no screen to click. No buttons. No forms. The two systems talk directly — machine to machine — through an API.

```
Your Agent / Your Code
    |
    |  no screen, no buttons, no human involved
    |  just code sending a structured request
    ▼
API
  (the contract that both systems understand)
    |
    ▼
MakeMyTrip Backend / Bank Backend / Hospital System
    |
    ▼
DATABASE
  (flights, accounts, patient records)
```

The API is designed for machines. It has endpoints (URLs), request formats (JSON), and response formats (JSON). A machine cannot use a frontend — it needs an API.

The Key Difference — Side by Side

```
HUMAN interacting with Zomato:
  Opens Zomato app
  Searches for "pizza near me"
  Clicks on a restaurant
  Adds items to cart
  Pays and orders
  → Uses the FRONTEND (the app)

AGENT interacting with Zomato:
  Calls search_restaurants("pizza", "Koramangala")
```

```
  Tool sends POST to https://api.zomato.com/v1/search
  Gets JSON response with restaurant list
  Calls place_order(restaurant_id, items)
  → Uses the API (no screen involved)

Same Zomato backend. Same database.
Different interface depending on who is talking.
```

Why This Matters for Agents

Your agent is not a human. It cannot open an app, cannot click a button, cannot fill a form. It can only send structured requests to an API and read structured responses back.

This is why **every tool you build for an agent is essentially an API call in disguise.**

```
Tool name the agent knows:    search_flights("Mumbai", "Goa", "May 5")
What the tool actually does:  calls MakeMyTrip API
What MakeMyTrip has for humans: a website with search boxes

Same data. Same backend.
Humans use the frontend.
Agents use the API.
```

Most large platforms — banks, hospitals, e-commerce companies, travel aggregators — have both:

```
A frontend   →  for their customers to use manually
An API       →  for other systems and agents to connect to programmatically
```

When you build an agent tool, you are using the API side of that platform.

The Restaurant Analogy

The best way to understand an API is through a restaurant.

```
You are a customer sitting at a table.          (Your application / your agent)
The kitchen is where the food is made.          (The other system — MakeMyTrip, bank, hospital)
The waiter is the API.                          (The API)

You do not walk into the kitchen yourself.
You do not know how the kitchen works.
You tell the waiter what you want.
The waiter goes to the kitchen, gets it, brings it back.

This is exactly how an API works:
  Your code says: "Give me flights from Mumbai to Goa on May 5"
  The API (waiter) takes this request to MakeMyTrip's servers (kitchen)
  MakeMyTrip processes it and gives the data back to the API
  The API brings the flight list back to your code
```

How Data Transfers Between Two Systems — Step by Step

This is exactly what happens every time your agent calls a tool that connects to an external service.

Step 1 — Your System Makes an HTTP Request

Your tool function sends an HTTP request to the other system's server. Every API call is basically an internet request — like visiting a website, but instead of getting HTML for a browser, you get data for your code.

```
Your tool sends this:

  POST https://api.makemytrip.com/v1/flights/search
  Headers:
    Authorization: Bearer mmt_api_key_abc123
    Content-Type: application/json
  Body:
    {
      "from": "BOM",
      "to": "GOI",
```

```
    "date": "2026-05-05",
    "class": "economy",
    "adults": 1
  }
```

This is a **POST request** — you are sending data to the server and asking it to process it.

Step 2 — The Request Travels Over the Internet

Your request leaves your server, travels through the internet (encrypted via HTTPS), and arrives at MakeMyTrip's servers — which could be anywhere in the world.

```
Your server  →  Internet (HTTPS encrypted)  →  MakeMyTrip server

The data is encrypted in transit.
Nobody in the middle can read it.
This is why HTTPS (not HTTP) is critical for APIs.
```

Step 3 — The Other System Processes the Request

MakeMyTrip's server receives your request. It:

- Verifies your API key (are you allowed to ask this?)
- Reads the parameters (from, to, date, class)
- Queries its own flight database
- Applies pricing and availability logic
- Assembles the response

```
MakeMyTrip's server:
  → checks API key: valid, belongs to partner account P-4821
  → queries flight database for BOM → GOI on May 5
  → finds 8 flights
  → returns top results with live prices
```

Step 4 — The Response Comes Back to Your System

The other system sends back a response — usually in JSON format. JSON is the universal language APIs use to exchange data.


```
{
  "status": "success",
  "flights": [
    {
      "flight_no": "6E-306",
      "airline": "IndiGo",
      "depart": "06:15",
      "arrive": "08:20",
      "price": 3240,
      "id": "FLT-6E306"
    },
    {
      "flight_no": "SG-158",
      "airline": "SpiceJet",
      "depart": "14:30",
      "arrive": "16:40",
      "price": 2980,
      "id": "FLT-SG158"
    }
  ]
}
```

Step 5 — Your Tool Reads the Response and Returns to the Agent

Your tool function reads this JSON response, formats it clearly, and returns it to the agent.

```
response = requests.post(url, json=params, headers=headers)
data = response.json()

result = []
for flight in data["flights"]:
```

```
    result.append(
        f"{flight['airline']} {flight['flight_no']} | "
        f"{flight['depart']} - {flight['arrive']} | "
        f"Rs {flight['price']} | ID: {flight['id']}"
    )
return "\n".join(result)
```

Agent receives:

```
IndiGo 6E-306 | 06:15 - 08:20 | Rs 3,240 | ID: FLT-6E306
SpiceJet SG-158 | 14:30 - 16:40 | Rs 2,980 | ID: FLT-SG158
```

The Complete API Data Transfer Picture

```
YOUR AGENT
    |
    |  calls tool
    ▼
YOUR TOOL FUNCTION
    |
    |  sends HTTP request (POST/GET)
    |  with: API key + parameters
    ▼
INTERNET (HTTPS encrypted)
    |
    ▼
EXTERNAL SERVER (MakeMyTrip / Bank / Hospital / Zomato)
    |
    |  validates API key
    |  processes the request
    |  queries its database
    |  builds the response
    ▼
INTERNET (HTTPS encrypted)
    |
    ▼
```

```
YOUR TOOL FUNCTION
    |
    |  reads JSON response
    |  formats as readable string
    ▼
YOUR AGENT
    |
    |  reads the result
    |  decides next step
    ▼
USER
```

The Four HTTP Methods — How Systems Talk

APIs use four main types of requests. You need to know these because every tool you write uses one of them.

Method	What It Does	Real Example
GET	Retrieve information — read only, no changes	Get account balance, search flights, get patient records
POST	Send data and create something	Book a flight, place an order, create a ticket
PUT	Update an existing record	Update patient details, edit a booking
DELETE	Remove a record	Cancel a booking, delete a file

```python
import requests

# GET — retrieve flight search results
response = requests.get(
    "https://api.amadeus.com/v2/shopping/flight-offers",
    params={"originLocationCode": "BOM", "destinationLocationCode": "GOI"},
    headers={"Authorization": "Bearer YOUR_KEY"}
)

# POST — book a flight
response = requests.post(
    "https://api.makemytrip.com/v1/flights/book",
```

```
    json={"flight_id": "FLT-6E306", "passenger": "Arjun Sharma"},
    headers={"Authorization": "Bearer YOUR_KEY"}
)

# DELETE — cancel a booking
response = requests.delete(
    "https://api.makemytrip.com/v1/bookings/BK-44821",
    headers={"Authorization": "Bearer YOUR_KEY"}
)
```

What Is a REST API

Most APIs you will use are **REST APIs** (Representational State Transfer). REST is simply a set of rules for how APIs should be designed so every developer can understand them.

```
REST rules (simplified):
  Use HTTP methods correctly (GET to read, POST to create)
  Use URLs that describe the resource:
    /flights/search          — search flights
    /bookings/BK-44821       — a specific booking
    /patients/P-4821/records — records for a specific patient
  Return data in JSON format
  Use status codes to indicate success or failure:
    200 — success
    201 — created successfully
    400 — bad request (wrong parameters)
    401 — unauthorized (wrong API key)
    404 — not found
    500 — server error
```

What Is an API Key and Why It Matters

An API key is like a **password for your application**. Every time your tool calls an external API, it includes the API key. The other system checks it to:

- Verify you are allowed to use this API
- Track how many calls you make (rate limiting)
- Bill you for usage
- Restrict what data you can access

```
Without API key:
  Your request: "Give me flights from Mumbai to Goa"
  Response: 401 Unauthorized. Access denied.

With valid API key:
  Your request: "Give me flights..." + "Bearer mmt_abc123"
  Response: Here are the flights.
```

Never put API keys directly in your code. Always use environment variables.

```python
# BAD — key exposed in code, visible to anyone who reads it
headers = {"Authorization": "Bearer sk-abc123realkey"}

# GOOD — key stored in environment, not in code
import os
headers = {"Authorization": f"Bearer {os.getenv('MMT_API_KEY')}"}
```

How One Real System Talks to Another — Complete Example

Arjun's logistics company wants its agent to automatically check fuel prices every morning and send a report. Here is the complete data flow:

```
Arjun's Agent
    |
    |  calls: get_fuel_price("petrol", "Mumbai")
    ▼
Tool Function
    |
    |  GET https://api.petrolprices.in/v1/current
    |  params: fuel_type=petrol, city=Mumbai
    |  headers: Authorization: Bearer PETROL_API_KEY
```

```
    ▼
petrolprices.in Server
    |
    |  validates key
    |  looks up Mumbai petrol price from IOCL data
    |  returns: {"price": 95.42, "city": "Mumbai", "date": "2026-05-05"}
    ▼
Tool Function
    |
    |  reads response
    |  returns: "Mumbai petrol price on May 5: Rs 95.42 per litre"
    ▼
Arjun's Agent
    |
    |  calls: calculate_fuel_cost("95.42", "12", "180")
    |  (price per litre, mileage, distance)
    ▼
Tool Function (no API needed — just maths)
    |
    |  returns: "Fuel cost for 180 km: Rs 1,431.30"
    ▼
Arjun's Agent
    |
    |  calls: send_email("arjun@company.com", "Daily Fuel Cost", "...")
    ▼
SendGrid API
    |
    |  delivers email to Arjun
    ▼
Agent responds: "Daily fuel cost report sent to your email."
```

Three different systems talked to each other — petrolprices.in, a local calculator, and SendGrid — all coordinated by the agent through tools. Arjun just asked one question and the agent handled the rest.

A tool is a **function that an agent can call** to interact with the real world.

Without tools, an agent can only do one thing — generate text from what it already knows. It cannot check today's flight prices. It cannot look up your bank balance. It cannot book a hotel room. It cannot send an email. It is just a text generator.

With tools, the agent becomes genuinely useful. It can search the web, call an API, read a database, run a calculation, send a notification, and take real actions in real systems.

> **In one sentence:** A tool is the bridge between the agent's intelligence and the real world.

The Analogy

Think of the agent as a brilliant consultant sitting in a room. The consultant is very knowledgeable but cannot do anything by themselves — they cannot open a browser, cannot pick up the phone, cannot access your company's database.

Tools are the **assistants** outside the room. The consultant says "check the flight prices for Mumbai to Delhi tomorrow" — the assistant goes and checks, comes back with the answer. The consultant reads it, thinks, and says "now book the cheapest one" — the assistant goes and books it.

The agent thinks. The tools act.

7.2 Why Tools Are the Most Important Part of an Agent

Without memory — the agent forgets. That is a limitation.
Without planning — the agent acts randomly. That is a problem.
Without tools — the agent cannot do anything real. That is a dealbreaker.

Every production agent you will ever build relies on tools. The quality of your tools determines the quality of your agent.

```
An agent without tools:
  User: "What is the cheapest flight from Mumbai to Goa tomorrow?"
  Agent: "I do not have access to real-time flight data.
          As of my training cutoff, airlines operating this route include..."

  USELESS.

An agent with tools:
  User: "What is the cheapest flight from Mumbai to Goa tomorrow?"
  Agent: [calls search_flights("Mumbai", "Goa", "tomorrow")]
  Tool returns: "IndiGo 6E-202 at Rs 2,840 | Air India AI-456 at Rs 3,150"
  Agent: "The cheapest option is IndiGo flight 6E-202 at Rs 2,840.
```

```
          Shall I book it for you?"

  USEFUL.
```

7.3 How Tool Calling Works — Step by Step

This is the exact sequence of events every time an agent calls a tool.

Step 1 — Agent Receives the User Request

The user sends a message. The agent reads it.

```
User: "Book me the cheapest flight from Bangalore to Mumbai
       for next Monday, aisle seat, and send the confirmation
       to my email."
```

Step 2 — Agent Reads the Available Tools

The agent has been given a list of tools with descriptions. It reads all tool names and descriptions to understand what it can do.

```
Available tools the agent can see:

  search_flights(origin, destination, date)
    "Search for available flights between two cities on a given date.
     Returns flight options with price, airline, and timings."

  book_flight(flight_id, seat_preference, passenger_details)
    "Book a specific flight. Returns booking confirmation and PNR."

  send_email(to_email, subject, body)
    "Send an email to a given address with subject and body."
```

```
  get_user_profile(user_id)
    "Retrieve user details including email, name, and saved preferences."
```

Step 3 — Agent Decides Which Tool to Call First

The agent thinks: "I need the user's email and name first before I can search or book anything." It decides to call `get_user_profile` first.

```
Agent reasoning:
  To book a flight I need: origin, destination, date, seat, email.
  I have: origin (Bangalore), destination (Mumbai), date (next Monday), seat (aisle).
  I am missing: user email and full name for the booking.
  First action: get_user_profile to retrieve email and name.
```

Step 4 — Agent Generates a Tool Call

The agent outputs a structured tool call — not a text response, but a JSON object with the tool name and parameters.

```
{
  "tool": "get_user_profile",
  "parameters": {
    "user_id": "USR-8821"
  }
}
```

Step 5 — Tool Runs and Returns Result

The system intercepts this tool call, runs the actual function, and passes the result back to the agent.

```
Tool result:
{
  "name": "Arjun Sharma",
```

```
  "email": "arjun.sharma@gmail.com",
  "frequent_flyer": "IndiGo BluChip",
  "saved_seat_preference": "aisle"
}
```

Step 6 — Agent Reads the Result and Decides Next Tool

The agent now has the user's email and name. It proceeds to search for flights.

```
{
  "tool": "search_flights",
  "parameters": {
    "origin": "Bangalore",
    "destination": "Mumbai",
    "date": "2026-05-05"
  }
}
```

```
Tool result:
  IndiGo 6E-306  |  06:15 - 08:20  |  Rs 3,240  |  ID: FLT-6E306
  Air India AI-619  |  09:00 - 11:10  |  Rs 3,890  |  ID: FLT-AI619
  SpiceJet SG-158  |  14:30 - 16:40  |  Rs 2,980  |  ID: FLT-SG158
```

Step 7 — Agent Calls the Booking Tool

Agent identifies SpiceJet as cheapest. Books it.

```
{
  "tool": "book_flight",
  "parameters": {
    "flight_id": "FLT-SG158",
    "seat_preference": "aisle",
    "passenger_name": "Arjun Sharma",
```

```
    "passenger_email": "arjun.sharma@gmail.com"
  }
}
```

```
Tool result:
  Booking confirmed. PNR: SG4821X
  Flight: SpiceJet SG-158
  Date: May 5, 2026 | 14:30 - 16:40
  Seat: 14C (aisle)
  Amount charged: Rs 2,980
```

Step 8 — Agent Sends the Confirmation Email

```
{
  "tool": "send_email",
  "parameters": {
    "to_email": "arjun.sharma@gmail.com",
    "subject": "Flight Booking Confirmed - SG-158 Bangalore to Mumbai",
    "body": "Dear Arjun, your flight is confirmed. PNR: SG4821X.
             SpiceJet SG-158 on May 5 at 14:30. Seat 14C (aisle).
             Amount: Rs 2,980."
  }
}
```

Step 9 — Agent Gives the Final Response to User

Only after all tools are done does the agent respond to the user.

```
Agent: "Done! I booked the cheapest flight for you:
        SpiceJet SG-158 on May 5 at 14:30, arriving Mumbai at 16:40.
        Aisle seat 14C. Total: Rs 2,980. PNR: SG4821X.
        Confirmation sent to arjun.sharma@gmail.com."
```

The Complete Tool Calling Loop

```
User message
     ↓
Agent reads tools list
     ↓
Agent decides: which tool do I need first?
     ↓
Agent generates tool call (name + parameters)
     ↓
System runs the tool
     ↓
Tool returns result
     ↓
Agent reads result → do I need another tool?
     ├── YES → generate next tool call → repeat
     └── NO  → write final response to user
```

This loop can run 1 time or 20 times depending on how complex the task is.

7.4 Types of Tools

Tools fall into four broad categories.

Type 1 — Search and Retrieval Tools

Used to find information from external sources.

```
web_search(query)              - search the internet
search_documents(query)        - search internal knowledge base
get_patient_records(id)        - retrieve from hospital database
search_flights(origin, dest)   - search flight availability
```

```
get_stock_price(ticker)        - fetch live market data
get_weather(city, date)        - retrieve weather forecast
```

Type 2 — Action and Transaction Tools

Used to take real actions in real systems.

```
book_flight(flight_id, pax)     - book a flight ticket
make_payment(amount, account)   - execute a bank transfer
send_email(to, subject, body)   - send an email
book_hotel(hotel_id, dates)     - confirm hotel reservation
cancel_order(order_id)          - cancel an e-commerce order
schedule_appointment(doc, time) - book a doctor's slot
```

How Action Tools Connect to Real Services

This is the most important thing to understand about action tools.

The agent never calls MakeMyTrip or OYO directly. The agent only calls the tool by name. Inside the tool function, you write the code that connects to the real API. The agent never sees the API details — it just sees the tool name and description.

```
What the agent does:
  book_flight("FLT-6E306", "Arjun Sharma")

What happens inside the tool:
  → calls MakeMyTrip Partner API
  → or calls IndiGo Direct Booking API
  → or calls Amadeus GDS (used by most travel companies)
  → whichever API your company has a contract with

What comes back to the agent:
  "Booking confirmed. PNR: 6E4821X. IndiGo 6E-306. Rs 3,240 charged."
```

The pattern is always the same three layers:

```
Layer 1 — Tool name       (what the agent calls)
                ↓
Layer 2 — Tool function   (Python code you write)
                ↓
Layer 3 — Real API        (MakeMyTrip, OYO, Razorpay, Practo...)
                ↓
Result travels back up the chain to the agent
```

You control which real API sits inside each tool. That is where your business partnerships and API keys matter. Here is exactly which real services sit behind each common action tool:

Tool Name	Real API / Service Behind It
`book_flight()`	MakeMyTrip Partner API, IndiGo API, Amadeus GDS, Skyscanner API
`book_hotel()`	OYO API, Airbnb API, Booking.com API, MakeMyTrip Hotels API
`make_payment()`	Razorpay API, PayU API, NPCI UPI API, Stripe API
`send_email()`	SendGrid API, AWS SES, Gmail SMTP API
`cancel_order()`	Flipkart Seller API, Amazon MWS API, Meesho API
`schedule_appointment()`	Practo API, Apollo Scheduling API, Hospital HMS
`book_cab()`	Ola API, Uber API, Rapido API
`order_food()`	Zomato API, Swiggy API
`transfer_funds()`	NPCI IMPS/NEFT API, Razorpay Transfer API
`send_sms()`	Twilio API, MSG91 API, AWS SNS

How to get API access in real life:

Most major services have a developer or partner program. You register, get API credentials (API key or OAuth token), and use them inside your tool function. The agent never knows or needs to know these credentials — they are handled entirely inside the tool code.

How to Get Real API Access — Service by Service

Getting API access is different for each service. Here is exactly what to do for the most common ones.

MakeMyTrip / Ixigo / Goibibo — Flight and Hotel APIs

These are B2B partner APIs — not public. You need a business agreement.

```
Step 1: Go to MakeMyTrip Partner Program
        URL: https://www.makemytrip.com/partner-program
        Register as a travel tech partner or affiliate

Step 2: Submit your business details
        Company name, GST number, use case, expected volume

Step 3: MakeMyTrip reviews your application (2-5 business days)

Step 4: On approval, you receive:
        API Key + Secret Key
        Sandbox URL for testing
        Production URL after testing is complete

Step 5: Use in your tool:
        headers = {"Authorization": "Bearer YOUR_API_KEY"}
```

For flight search without a partner agreement, use **Amadeus API** (amadeus.com/en) — it is publicly available with a free tier for developers and covers most Indian and international routes.

OYO / Airbnb — Hotel Booking APIs

```
OYO:
  URL: https://www.oyorooms.com/partners
  OYO has a Partner Extranet API for hotels and travel aggregators
  Register as a technology partner → get sandbox credentials → go live

Airbnb:
  URL: https://www.airbnb.com/partner
  Airbnb Connect API is available for property management systems
  Apply via their partner portal → review takes 1-2 weeks
```

```
Alternative for hotel search (no partnership needed):
  RapidAPI → search "Hotels" → Booking.com API, Tripadvisor API
  URL: https://rapidapi.com
  Sign up free, subscribe to hotel APIs, get API key instantly
```

Razorpay / PayU — Payment APIs

These are publicly available with just a business account.

```
Razorpay:
  URL: https://razorpay.com/docs/api
  Step 1: Create a Razorpay account at razorpay.com
  Step 2: Go to Dashboard → Settings → API Keys
  Step 3: Generate Key ID and Key Secret
  Step 4: Use in your tool immediately
  Free sandbox available. Go live after KYC verification.

PayU:
  URL: https://developer.payumoney.com
  Same process — create account, get merchant key and salt
  Sandbox available for testing without real money
```

SendGrid / AWS SES — Email APIs

The easiest APIs to get access to — free tiers available immediately.

```
SendGrid:
  URL: https://sendgrid.com
  Step 1: Sign up for free account
  Step 2: Go to Settings → API Keys → Create API Key
  Step 3: Copy the key — use it in your tool
  Free tier: 100 emails per day
  Paid: from $19.95/month for higher volumes
```

```
AWS SES (Simple Email Service):
  URL: https://aws.amazon.com/ses
  Step 1: Create AWS account at aws.amazon.com
  Step 2: Go to SES in AWS Console → Create Identity (your domain or email)
  Step 3: Go to IAM → Create user → Attach SES policy → Download Access Key
  Step 4: Use Access Key in your tool
  Cost: $0.10 per 1,000 emails
```

Practo / Apollo — Healthcare Appointment APIs

These are restricted APIs — for healthcare businesses only.

```
Practo:
  URL: https://platform.practo.com
  Practo Platform API is for clinics, hospitals, and health apps
  Apply via their developer portal with healthcare business proof
  On approval: get API key + clinic/doctor database access

Apollo:
  Apollo 247 has an API for corporate health programs
  Contact: partnerships@apollo247.com
  Enterprise agreement required

Alternative for development and testing:
  Build your own mock doctor database
  Use Google Calendar API for appointment scheduling logic
  URL: https://developers.google.com/calendar
```

Zomato / Swiggy — Food Delivery APIs

```
Zomato:
  URL: https://developers.zomato.com/api
  Zomato API gives access to restaurant data, menus, and reviews
  Sign up → get API key instantly (free for restaurant search)
  Note: Actual order placement API is for registered restaurant partners only
```

```
Swiggy:
  Swiggy does not have a public API
  Access is only for registered restaurant partners
  Apply via: https://partner.swiggy.com

Alternative:
  For development, use Zomato API for restaurant data
  Simulate order placement with your own mock API
```

Ola / Uber — Cab Booking APIs

```
Ola:
  URL: https://developers.olacabs.com
  Register as a developer → get API key
  Sandbox available for testing ride booking flow

Uber:
  URL: https://developer.uber.com
  Create developer account → create an app → get Client ID and Secret
  Free sandbox with simulated rides
```

MSG91 / Twilio — SMS APIs

The fastest APIs to get — live within minutes.

```
MSG91 (Indian service, recommended for India):
  URL: https://msg91.com/api
  Step 1: Sign up at msg91.com
  Step 2: Go to API → Get API Key
  Step 3: Buy credits (starts at Rs 500 for 1,000 SMS)
  Step 4: Use API key in your tool immediately

Twilio (global):
  URL: https://www.twilio.com/try-twilio
```

```
  Step 1: Sign up for free trial
  Step 2: Get Account SID and Auth Token from dashboard
  Step 3: Free trial includes a small credit for testing
  Paid: around $0.0075 per SMS in India
```

NPCI — UPI / NEFT / IMPS APIs

These are restricted to licensed financial institutions.

```
NPCI (National Payments Corporation of India):
  URL: https://www.npci.org.in/what-we-do/upi/product-overview
  Access requires:
    RBI licence or partnership with a licensed bank
    Technical certification from NPCI
    Security audit and compliance review

For startups and fintechs:
  Use a payment aggregator instead — Razorpay, Cashfree, or Paytm
  They sit on top of NPCI and give you a simpler API
  Razorpay: razorpay.com/docs/payments/upi
  Cashfree: cashfree.com/devstudio
```

The Quick Summary — How to Get API Access

Service	Where to Register	Time to Get Access	Cost
Amadeus (flights)	amadeus.com/developers	Instant	Free sandbox
RapidAPI hotels	rapidapi.com	Instant	Free tier available
Razorpay	razorpay.com	1-2 days (KYC)	Free sandbox, pay per transaction
SendGrid	sendgrid.com	Instant	Free 100 emails/day
AWS SES	aws.amazon.com	Instant	Pay per email
Twilio (SMS)	twilio.com	Instant	Free trial credit

Service	Where to Register	Time to Get Access	Cost
MSG91 (SMS)	msg91.com	Instant	Rs 500 for 1,000 SMS
Ola	developers.olacabs.com	1-2 days	Free sandbox
Uber	developer.uber.com	Instant	Free sandbox
MakeMyTrip	Partner program	5-7 days	Business agreement
Practo	platform.practo.com	3-5 days	Healthcare business required
OYO	oyorooms.com/partners	3-5 days	Partner agreement

The General Pattern for Any API

Whatever service you want to connect to, the process is always the same:

```
Step 1 — Find the developer portal
          Search: "[service name] API" or "[service name] developer"
          e.g. "Zomato API", "OYO developer", "Practo API"

Step 2 — Create an account or apply for access
          Public APIs: instant signup, get key immediately
          Partner APIs: fill a form, wait for review

Step 3 — Get your credentials
          Usually: API Key + Secret
          Sometimes: Client ID + Client Secret (OAuth)
          Sometimes: Username + Password

Step 4 — Store credentials securely
          NEVER put API keys in your code directly
          Use environment variables:
          os.environ["RAZORPAY_KEY"] = "your_key"
          key = os.getenv("RAZORPAY_KEY")

Step 5 — Test in sandbox first
          Every serious API has a sandbox/test environment
          Test with fake data before going to production
```

```
Step 6 — Put the API call inside your tool function
          Agent calls the tool → tool calls the API
          Agent never sees the credentials
```

```python
# Example: book_flight tool connecting to MakeMyTrip API
import requests

@tool
def book_flight(flight_id: str, passenger_name: str, passenger_email: str) -> str:
    """
    Book a flight ticket for a passenger.
    Use this after the user has confirmed which flight they want to book.
    """
    response = requests.post(
        "https://api.makemytrip.com/v2/flights/book",
        json={
            "flight_id": flight_id,
            "passenger": {"name": passenger_name, "email": passenger_email}
        },
        headers={"Authorization": "Bearer MMT_API_KEY_HERE"}
        # API key stored securely, never exposed to the agent
    )
    if response.status_code == 200:
        data = response.json()
        return f"Booking confirmed. PNR: {data['pnr']}. Amount: Rs {data['amount']}."
    else:
        return "Booking failed. Please try again or contact support."
```

The agent called `book_flight()`. MakeMyTrip's servers processed the booking. The agent received a confirmation. At no point did the agent know it was talking to MakeMyTrip — it just called the tool.

Type 3 — Computation and Analysis Tools

Used to calculate, process, or analyse data.

```
calculate(expression)          — evaluate a math expression
run_sql_query(query)           — query a database
analyse_csv(file_path)         — analyse a data file
check_drug_interaction(drugs)  — medical safety check
calculate_emi(principal, rate) — loan EMI calculator
currency_convert(amount, from) — currency conversion
```

Type 4 — Communication and Notification Tools

Used to send messages and alerts.

```
send_sms(phone, message)       — send SMS notification
send_whatsapp(phone, message)  — WhatsApp message
create_ticket(issue, priority) — raise a support ticket
send_push_notification(user)   — mobile push alert
post_to_slack(channel, msg)    — Slack message
create_calendar_event(details) — schedule a meeting
```

7.5 Real-World Tools by Industry

This section shows exactly which real-world APIs and services act as tools for agents in each industry.

Travel and Flight Booking

A travel planning agent needs to search flights, compare prices, book tickets, arrange hotels, and plan the itinerary. Each of these is a separate tool pointing to a real service.

Tool Name	Real Service Behind It	What It Does
search_flights	MakeMyTrip API, Ixigo API, Skyscanner API	Search available flights with prices
book_flight	IndiGo Partner API, Air India API, SpiceJet API	Confirm booking and generate PNR

Tool Name	Real Service Behind It	What It Does
search_hotels	OYO API, Airbnb API, Booking.com API	Search hotels by city and dates
book_hotel	OYO API, MakeMyTrip Hotels API	Confirm hotel reservation
get_weather	OpenWeatherMap API, IMD API	Weather forecast for travel dates
plan_itinerary	Google Maps API, TripAdvisor API	Suggest places and route
currency_convert	RBI API, Fixer.io API	Convert currency for international travel
send_confirmation	SendGrid API, AWS SES	Email booking confirmation

Example agent conversation:

```
User: "Plan a 4-day trip to Goa from Bangalore next weekend.
       Budget Rs 20,000 for flights and hotel."

Agent calls: search_flights("Bangalore", "Goa", "Fri May 8")
Agent calls: search_flights("Goa", "Bangalore", "Mon May 11")
Agent calls: search_hotels("Goa", "May 8", "May 11", max_price=8000)
Agent calls: get_weather("Goa", "May 8 to May 11")
Agent calls: plan_itinerary("Goa", 4 days, interests="beach, food")
Agent responds: complete 4-day itinerary within budget
```

Banking and Finance

A banking agent handles account queries, fund transfers, loan applications, and investment advice. Every action touches a real banking API.

Tool Name	Real Service Behind It	What It Does
get_account_balance	Core Banking API (Finacle, BankWare)	Retrieve current balance
get_transaction_history	Bank Statement API	Last N transactions
transfer_funds	NEFT/IMPS/UPI API (NPCI)	Transfer money between accounts
check_loan_eligibility	CIBIL API, Experian API	Check credit score and loan eligibility
apply_for_loan	Bank Loan Origination System	Submit loan application

Tool Name	Real Service Behind It	What It Does
get_fd_rates	Bank Product API	Fixed deposit interest rates
block_card	Card Management API (Visa, Mastercard)	Block lost or stolen card
raise_dispute	Dispute Management System	Raise transaction dispute
get_emi_details	Loan Management API	EMI schedule and outstanding
calculate_emi	Internal calculation	EMI for given principal and rate

Example agent conversation:

```
User: "I want to transfer Rs 50,000 to my brother's account
       and then check if I am eligible for a personal loan of Rs 3 lakhs."

Agent calls: get_account_balance("ACC-882211")
             → Balance: Rs 1,24,500

Agent calls: transfer_funds("ACC-882211", "ACC-441892", 50000, "IMPS")
             → Transfer successful. Ref: IMPS2024882

Agent calls: check_loan_eligibility("CUS-882211", 300000)
             → CIBIL Score: 748 | Eligible for up to Rs 5 lakh at 11.5% p.a.

Agent: "Transfer of Rs 50,000 done. Reference IMPS2024882.
        Good news — you are eligible for a personal loan up to Rs 5 lakh
        at 11.5% per annum. Shall I start the application?"
```

Healthcare

A healthcare agent helps patients book appointments, checks symptoms, retrieves medical records, and prepares discharge summaries.

Tool Name	Real Service Behind It	What It Does
search_doctors	Practo API, Apollo API	Find doctors by speciality and location
book_appointment	Hospital Scheduling System	Book a consultation slot

Tool Name	Real Service Behind It	What It Does
get_patient_records	EHR System (Epic, Cerner)	Retrieve medical history
check_vitals	IoT Medical Devices API	Real-time vitals monitoring
check_drug_interaction	DrugBank API, Rx-Nav API	Drug safety check
get_lab_results	Lab Information System	Blood test and diagnostic results
check_insurance	Insurance TPA API	Verify coverage and benefits
raise_insurance_claim	Claims Management System	Submit hospital bill claim
send_prescription	Pharmacy Management System	Send prescription to pharmacy
schedule_followup	Doctor Calendar API	Book follow-up appointment

Example agent conversation:

```
User: "I have knee pain for 3 days. Find me an orthopaedic doctor
       near Koramangala Bangalore and book the earliest appointment."

Agent calls: search_doctors("orthopaedic", "Koramangala", "Bangalore")
             → Dr. Ramesh Nair, Apollo Clinic — available Today 4PM
             → Dr. Priya Menon, Fortis — available Tomorrow 10AM

Agent calls: check_insurance("PAT-44821", "outpatient_consultation")
             → Covered under Mediclaim. Co-pay: Rs 200.

Agent calls: book_appointment("DR-NAIR-APOLLO", "Today", "4PM", "PAT-44821")
             → Confirmed. Appointment ID: APT-20240501-884

Agent: "Booked with Dr. Ramesh Nair at Apollo Koramangala today at 4 PM.
        Your insurance covers it. Co-pay only Rs 200.
        Confirmation sent to your phone."
```

E-Commerce

An e-commerce agent helps customers search products, track orders, process returns, and manage their account.

Tool Name	Real Service Behind It	What It Does
search_products	Amazon API, Flipkart API, Meesho API	Search and filter products
get_product_details	Product Catalogue API	Price, specs, reviews
check_availability	Inventory Management API	Stock and delivery timeline
place_order	Order Management System	Confirm purchase
track_order	Shiprocket API, Delhivery API	Real-time shipment tracking
cancel_order	Order Management System	Cancel before dispatch
initiate_return	Returns Management System	Start return or exchange
process_refund	Payment Gateway API	Refund to original payment method
apply_coupon	Promotions Engine	Validate and apply discount code
get_recommendations	Recommendation Engine	Personalised product suggestions

Example agent conversation:

```
User: "My order OR-44821 for a Nike shoe hasn't arrived yet.
       It was supposed to come 3 days ago."

Agent calls: track_order("OR-44821")
             → Status: Delayed. Last update: In transit at Pune hub.
             → Expected new delivery: Tomorrow by 8PM.

Agent calls: get_order_details("OR-44821")
             → Nike Air Max, Size 9. Ordered on April 28.
             → Delivery promised: May 1. Current date: May 4.

Agent: "Your Nike shoe is delayed and still in transit at the Pune hub.
        New estimated delivery is tomorrow by 8 PM.
        Would you like me to raise a complaint or wait till tomorrow?"
```

HR and Enterprise

An HR agent helps employees apply for leave, check payslips, raise IT tickets, and manage onboarding.

Tool Name	Real Service Behind It	What It Does
get_payslip	HRMS API (SAP, Workday)	Retrieve monthly payslip
apply_leave	Leave Management System	Submit leave application
check_leave_balance	HRMS API	Remaining leave count
raise_it_ticket	ServiceNow API, Jira API	Log IT support request
book_meeting_room	Facilities Management API	Reserve conference room
get_policy	Knowledge Management System	Retrieve HR policy document
onboard_employee	HRMS + IT Provisioning	Set up accounts for new joiner
get_org_chart	Directory API	Find reporting structure
submit_expense	Expense Management System	Submit travel or business expense
get_training_courses	LMS API (Cornerstone, Moodle)	Available training programs

---

Food Delivery

A food delivery agent takes orders, tracks them, and handles complaints.

Tool Name	Real Service Behind It	What It Does
search_restaurants	Zomato API, Swiggy API	Restaurants near location
get_menu	Restaurant Menu API	Full menu with prices
place_order	Order Management System	Confirm food order
track_delivery	GPS Tracking API	Real-time delivery partner location
apply_promo	Promotions Engine	Apply discount or cashback
rate_order	Review System	Submit rating and review
reorder	Order History API	Repeat a previous order
cancel_order	Order System	Cancel if not yet prepared

7.6 How to Give an Agent Tool Access

This is the most practical section. Here is exactly how tool access works in code — from writing the tool to giving it to an agent.

Step 1 — Write the Tool Function

Every tool is a Python function with a clear description. The description is what the LLM reads to decide when to use this tool.

```
from langchain_core.tools import tool

@tool
def get_account_balance(account_id: str) -> str:
    """
    Retrieve the current account balance for a given account ID.
    Use this when the user asks about their balance, funds, or account status.
    Returns the available balance in Indian Rupees.
    """
    # In real life this calls the bank's core banking API
    # Here we simulate it
    balances = {
        "ACC-882211": "Available balance: Rs 1,24,500",
        "ACC-441892": "Available balance: Rs 34,200",
        "ACC-993341": "Available balance: Rs 8,76,000",
    }
    return balances.get(account_id, "Account not found. Please check the account ID.")
```

The docstring is critical. The LLM reads it to understand:

- What this tool does
- When to use it
- What input it needs
- What output to expect

Step 2 — Connect to a Real API (Production Pattern)

In production, the tool calls a real external API instead of a mock dictionary.

```
import requests

@tool
def search_flights(origin: str, destination: str, date: str) -> str:
    """
    Search for available flights between two cities on a given date.
    Input: origin city, destination city, date in YYYY-MM-DD format.
    Returns a list of flights with airline, timing, and price.
    """
    # Real API call to MakeMyTrip or Ixigo or Amadeus
    response = requests.get(
        "https://api.makemytrip.com/v1/flights/search",
        params={
            "from": origin,
            "to": destination,
            "date": date,
            "class": "economy"
        },
        headers={"Authorization": "Bearer YOUR_API_KEY"}
    )

    if response.status_code == 200:
        flights = response.json()["flights"]
        result = []
        for f in flights[:5]:  # top 5 results
            result.append(
                f"{f['airline']} {f['flight_no']} | "
                f"{f['depart_time']} - {f['arrive_time']} | "
                f"Rs {f['price']}"
            )
        return "\n".join(result)
    else:
        return "Flight search unavailable. Please try again."
```

Step 3 — Give the Tools to the Agent

In LangChain / LangGraph:

```
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

# Bind tools to the LLM
tools = [get_account_balance, search_flights, transfer_funds, send_email]
llm_with_tools = llm.bind_tools(tools)

# The LLM now knows about all 4 tools
# It will call them when relevant
```

In CrewAI:

```
from crewai import Agent

banking_agent = Agent(
    role="Personal Banking Assistant",
    goal="Help customers with account queries, transfers, and banking services.",
    backstory="You are a helpful bank assistant with access to real banking tools.",
    tools=[get_account_balance, transfer_funds, check_loan_eligibility],
    # only this agent gets banking tools
)

travel_agent = Agent(
    role="Travel Planning Assistant",
    goal="Find and book the best travel options for the customer.",
    backstory="You are an expert travel planner with access to flight and hotel booking tools.",
    tools=[search_flights, book_flight, search_hotels, book_hotel],
    # only this agent gets travel tools
)
```

In AutoGen:

```
from autogen import AssistantAgent

travel_assistant = AssistantAgent(
    name="TravelAgent",
    system_message="You are a travel planning assistant. Use the tools provided to search and book flights and hotels.",
    llm_config={
        "config_list": [{"model": "gpt-3.5-turbo", "api_key": "..."}],
        "functions": [
            {
                "name": "search_flights",
                "description": "Search for available flights",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "origin": {"type": "string"},
                        "destination": {"type": "string"},
                        "date": {"type": "string"}
                    }
                }
            }
        ]
    }
)
```

Step 4 — How the Agent Decides Which Tool to Call

The agent never calls tools randomly. It follows this logic:

```
1. Read the user's message carefully
2. Read all tool names and descriptions
3. Ask: "Which tool do I need to answer this?"
4. Ask: "Do I have all the parameters to call that tool?"
   If YES → call the tool
   If NO  → ask the user for the missing information
5. Read the tool result
6. Ask: "Do I need another tool to complete the task?"
```

```
    If YES → call the next tool
    If NO  → write the final response
```

Step 5 — Tool Access Security

Not every tool should be available to every agent. In production you control this carefully.

```
Banking agent tools:
  get_account_balance      – allowed
  transfer_funds           – allowed (with limit check)
  apply_for_loan           – allowed
  book_flight              – NOT allowed (wrong domain)
  delete_database          – NEVER allowed

Tool-level safeguards:
  transfer_funds checks:
    – Amount <= daily limit (Rs 2,00,000 per UPI rules)
    – Account is not blocked
    – User is authenticated (OTP verified)
    – Destination account is valid

  book_appointment checks:
    – Doctor is available at that time
    – Patient's insurance is active
    – No duplicate booking
```

7.7 Tool Design Best Practices

These rules make the difference between a tool that works reliably and one that confuses the agent.

Rule 1 — Write a clear, specific docstring

The docstring is what the LLM reads. Make it specific about what the tool does, when to use it, and what it returns.

```
# BAD — too vague
def get_data(id: str) -> str:
    """Get data for an ID."""

# GOOD — specific and complete
def get_patient_records(patient_id: str) -> str:
    """
    Retrieve the medical history for a patient.
    Use this when you need to know a patient's past diagnoses,
    medications, allergies, or treatment history.
    Input: patient ID in format P-XXXX (e.g., P-4821).
    Returns: diagnosis history, current medications, and allergies.
    """
```

Rule 2 — Return human-readable strings, not raw JSON

The agent reads the tool output as text. Format it clearly.

```
# BAD — agent has to parse complex JSON
return {"flights": [{"id": "FLT-001", "price": 3240, ...}]}

# GOOD — agent can read this directly
return (
    "IndiGo 6E-306 | 06:15 to 08:20 | Rs 3,240 | ID: FLT-6E306\n"
    "SpiceJet SG-158 | 14:30 to 16:40 | Rs 2,980 | ID: FLT-SG158"
)
```

Rule 3 — Handle errors gracefully

When a tool fails, return a helpful message — not an exception.

```python
def transfer_funds(from_account: str, to_account: str, amount: float) -> str:
    try:
        # call the banking API
        result = banking_api.transfer(from_account, to_account, amount)
        return f"Transfer successful. Reference: {result['ref_id']}"
    except InsufficientFundsError:
        return "Transfer failed: Insufficient balance in account."
    except DailyLimitExceededError:
        return "Transfer failed: Daily transfer limit of Rs 2,00,000 exceeded."
    except Exception as e:
        return f"Transfer failed due to a system error. Please try again."
```

Rule 4 — One tool, one job

Do not create a tool that does multiple things. Split it.

```python
# BAD — does too much
def book_travel(flight_id, hotel_id, cab_id):
    ...

# GOOD — each tool has one job
def book_flight(flight_id, passenger_details): ...
def book_hotel(hotel_id, check_in, check_out): ...
def book_cab(pickup, drop, date): ...
```

Rule 5 — Give each agent only the tools it needs

An agent does not need access to all tools. Give it only what is relevant to its role.

```python
# Medical Reviewer only needs medical tools
medical_reviewer = Agent(
    tools=[get_patient_records, check_vitals, check_drug_interaction]
)
```

```
# Billing Agent only needs billing tools
billing_agent = Agent(
    tools=[get_insurance_details, calculate_bill, raise_claim]
)
```

7.8 Tool Calling in LangGraph, AutoGen, and CrewAI

Each framework handles tool calling slightly differently.

In LangGraph

Tools are bound to the LLM. The agent node calls the LLM with tools bound. If the LLM requests a tool, a separate tool node runs it. The result goes back to the agent node via state.

```
llm_with_tools = llm.bind_tools(tools)

def agent_node(state):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

def tool_node(state):
    last = state["messages"][-1]
    results = []
    for tc in last.tool_calls:
        result = tool_map[tc["name"]].invoke(tc["args"])
        results.append(ToolMessage(content=result, tool_call_id=tc["id"]))
    return {"messages": results}

# Loop: agent -> tool -> agent -> tool -> agent -> done
```

In AutoGen

Tools are passed as function definitions in the llm_config. AutoGen handles the call-and-return loop automatically in the conversation.

```
llm_config = {
    "functions": [
        {
            "name": "search_flights",
            "description": "Search flights between two cities",
            "parameters": {...}
        }
    ]
}
```

In CrewAI

Tools are attached to agents using the `tools` parameter. CrewAI handles calling them when the agent decides to use them during task execution.

```
agent = Agent(
    role="Travel Assistant",
    tools=[search_flights, book_flight],
    ...
)
```

7.9 Section Summary

Tools are what make agents real. Without tools, an agent is just a text generator. With the right tools, an agent can book flights, transfer money, retrieve medical records, track orders, and take real actions in real systems.

The Tool Calling Loop in One Diagram

```
User request
     ↓
```

```
Agent reads tool descriptions
     ↓
Agent decides: which tool first?
     ↓
Agent generates tool call (name + parameters)
     ↓
System runs the tool (real API, database, function)
     ↓
Tool returns result
     ↓
Agent reads result → need another tool?
     ├── YES → generate next tool call
     └── NO  → write final response to user
```

Tools by Industry — Quick Reference

Industry	Key Tools
Travel	search_flights, book_flight, search_hotels, book_hotel, get_weather
Banking	get_balance, transfer_funds, check_eligibility, apply_loan, block_card
Healthcare	search_doctors, book_appointment, get_records, check_drug_interaction
E-Commerce	search_products, place_order, track_order, process_return, apply_coupon
HR	get_payslip, apply_leave, raise_ticket, book_room, submit_expense
Food Delivery	search_restaurants, place_order, track_delivery, cancel_order

---

The Three Rules of Good Tools

One job per tool. A tool that does multiple things confuses the agent and is hard to debug. Split booking, searching, and cancelling into separate tools.

Write the docstring for the LLM, not for humans. The LLM reads the docstring to decide when and how to call the tool. Be specific — mention what input format is expected and what the output looks like.

Never let a tool crash the agent. Wrap every API call in a try-except block. Return a clear error message as a string so the agent can handle it gracefully and inform the user.

Suresh D R — AI Product Developer & Technology Mentor
Agentic AI — Section 7 Tool Calling | Generative AI Training Course